

PyTorch

Motivación

- ¿Qué **librerías** existen para *deep learning*? ¿Cuál vamos a utilizar?
- ¿Cuál es la **estructura de datos** que se utiliza? ¿Cuáles son sus **operaciones**?

Organización

Etapas	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:45	05'
PyTorch	08:05	09:50	10'
Tensores	08:15	10:00	25'
Taller Intervalo de práctica	08:40	10:35	35'
Taller Puesta en común	09:15	11:10	20'
Fin de la clase	09:35	11:20	-

PyTorch | Qué es



PyTorch es una librería **open source** de deep learning, desarrollada para ser **flexible** y **modular** para la investigación, pero con la **estabilidad** y el **soporte** necesarios para su uso en producción.

Está basada en **Python** y fue diseñada para realizar cálculos numéricos haciendo uso de **tensores**.

Además, permite su ejecución en **GPU** para **acelerar los cálculos**.

PyTorch | Opciones



theano

The PyTorch logo, consisting of an orange flame-like icon with a dot at the top, followed by the word 'PyTorch' in a large, black, sans-serif font.

PyTorch



PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.*
- Fácil de **aprender**.*
- **Debugging** sencillo.*

* Al estar integrado completamente con Python y NumPy, PyTorch:

- Tiene una sintaxis muy similar a la de NumPy, por lo cual logra que el aprendizaje sea más sencillo respecto a otras librerías.
- Se puede “debuggear” con herramientas de debugging de Python.

PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.
- Fácil de **aprender**.
- ***Debugging*** sencillo.
- ***Data parallelism***.*

* PyTorch posee paralelismo de datos. Con esta función, puede distribuir el trabajo computacional entre múltiples núcleos de CPU o GPU.

PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.
- Fácil de **aprender**.
- ***Debugging*** sencillo.
- ***Data parallelism***.
- Soporte de **grafos dinámicos**.*

** PyTorch admite grafos computacionales dinámicos, lo que significa que el comportamiento de la red se puede cambiar mediante programación en tiempo de ejecución. Al tener un enfoque dinámico, se pueden ver todos los cálculos y saber exactamente lo que está sucediendo.*

PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.
- Fácil de **aprender**.
- ***Debugging*** sencillo.
- ***Data parallelism***.
- Soporte de **grafos dinámicos**.
- Proporciona un nuevo ***front-end híbrido***.*

* Hay dos modos de operación: el modo “eager” y el modo “graph”. El modo eager se suele utilizar para la investigación y el desarrollo, porque proporciona flexibilidad y facilidad de uso. Mientras tanto, el modo “graph” se suele usar para la producción, ya que proporciona mejor velocidad, optimización y funcionalidad.

PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.
- Fácil de **aprender**.
- ***Debugging*** sencillo.
- ***Data parallelism***.
- Soporte de **grafos dinámicos**.
- Proporciona un nuevo ***front-end híbrido***.
- Soporte nativo de **ONNX**.*

** ONNX: Open Neural Network Exchange. Puede exportar modelos en este formato. Esto permite que los modelos basados en PyTorch accedan directamente a las plataformas y tiempos de ejecución compatibles con ONNX.*

PyTorch | Por qué | En general

- Integrado completamente con **Python** y **NumPy**.
- Fácil de **aprender**.
- **Debugging** sencillo.
- **Data parallelism**.
- Soporte de **grafos dinámicos**.
- Proporciona un nuevo **front-end híbrido**.
- Soporte nativo de **ONNX**.
- Bien recibido por las principales plataformas en la **nube**.*

** Permite realizar trabajos de capacitación a gran escala en GPU y proporciona la capacidad de ejecutar modelos en un entorno de producción.*

PyTorch | Por qué | Relativo a TensorFlow

Comparison Factors	TensorFlow	PyTorch
Ramp-Up Time		<i>W</i>
Graph Creation And Debugging		<i>W</i>
Coverage	<i>W</i>	
Serialization	<i>W</i>	
Deployment	<i>W</i>	
Documentation	<i>Tie</i>	<i>Tie</i>
Device Management	<i>W</i>	
Custom Extensions		<i>W</i>

Más información disponible [acá](#).

PyTorch | Dónde

¿Dónde podemos encontrar recursos útiles sobre PyTorch?

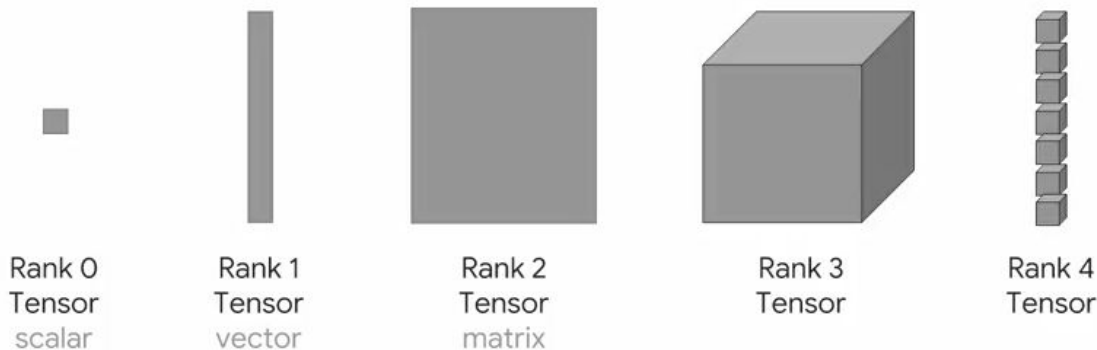
Primera opción: [documentación oficial](#).

Cheat sheet: [Stefan Seegerer](#).

Tutoriales: [oficiales](#) y [Tutorials Point](#).

Tensores | Concepto

Los **tensores** son una **estructura de datos** especializada, que se utiliza en PyTorch, para codificar los **inputs**, los **outputs** y los **parámetros** de un modelo.



Son **similares** a los **arrays** de n dimensiones de **NumPy**, con la diferencia de que los tensores también pueden utilizarse en una **GPU** para **acelerar los cálculos**.

Tensores | Modo

Para conocer qué operaciones podemos hacer con los tensores, veamos `td6-p14-c-tensores.ipynb`.

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
3. Calcular las **predicciones**: $\hat{y} = \sigma(z)$

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
3. Calcular las **predicciones**: $\hat{y} = \sigma(z)$
4. Aplicar la función de **costo**: $J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
3. Calcular las **predicciones**: $\hat{y} = \sigma(z)$
4. Aplicar la función de **costo**: $J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$
5. Aplicar noción del **gradiente** descendente: $\frac{\partial J}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m \left[(y^{(i)} - \hat{y}^{(i)}) x_j^{(i)} \right]$

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
3. Calcular las **predicciones**: $\hat{y} = \sigma(z)$
4. Aplicar la función de **costo**: $J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$
5. Aplicar noción del **gradiente** descendente: $\frac{\partial J}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m \left[(y^{(i)} - \hat{y}^{(i)}) x_j^{(i)} \right]$
6. **Actualizar** los **parámetros**: $w_j = w_j - \alpha \frac{\partial J}{\partial w_j}$

Taller | Repaso de regresión logística

Repasemos el **algoritmo de regresión logística**:

1. **Inicializar** los **parámetros**: $w_0, w_1, \dots, w_n$
2. Aplicar la función **logística**: $\sigma(z) = \frac{1}{1 + e^{-z}}$ donde $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$.
3. Calcular las **predicciones**: $\hat{y} = \sigma(z)$
4. Aplicar la función de **costo**: $J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$
5. Aplicar noción del **gradiente** descendente: $\frac{\partial J}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m \left[(y^{(i)} - \hat{y}^{(i)}) x_j^{(i)} \right]$
6. **Actualizar** los **parámetros**: $w_j = w_j - \alpha \frac{\partial J}{\partial w_j}$
7. **Repetir** los pasos 2 a 6 hasta alcanzar la **convergencia** o el **número máximo de iteraciones**.

Taller | Consigna

Implementemos una regresión logística utilizando sólo tensores de PyTorch. Para eso veamos `td6-p14-c-taller-consignas.ipynb`.

Taller | Resolución

Para conocer la resolución, veamos `td6-p14-d-taller-resoluciones.ipynb`.

Cierre

Hoy vimos y practicamos, entre otras cuestiones,

- qué **librerías** existen para **deep learning** y cuál vamos a utilizar,
- cuál es la **estructura de datos** que se utiliza y cuáles son sus **operaciones** y

Pueden darnos *feedback* de la clase [acá](#).